

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Môn: Cơ Sở Dữ Liệu Phân Tán | Giảng viên: Lê Hà Thanh

BÁO CÁO PHÂN TÍCH HỆ THỐNG

Hybrid Document-Graph Store: Medical Knowledge Base

Trần Hữu Trung – N23DCCN200 – D23CQCN03-N

Section 1: Giới Thiệu Lý Thuyết Özsu & Valduriez

Özsu và Valduriez (2020) định nghĩa một hệ cơ sở dữ liệu phân tán (DDBMS) là một tập hợp các cơ sở dữ liệu logic liên quan nhau phân bố trên một mạng máy tính, được quản lý bởi DDBMS sao cho người dùng nhìn thấy như một hệ thống thống nhất duy nhất. Lý thuyết này là nền tảng cho toàn bộ thiết kế của đề tài.

1.1. Mười hai quy tắc của Date cho DDBMS

Date (1987) đề xuất 12 quy tắc cho DDBMS, trong đó quan trọng nhất đối với đề tài bao gồm:

- Local autonomy: Mỗi phân vùng (partition) trong hệ thống hoạt động độc lập, không phụ thuộc vào các phân vùng khác.
- Location transparency: Người dùng không cần biết dữ liệu nằm ở phân vùng nào.
- Fragmentation transparency: Người dùng thấy dữ liệu như một tập hợp thống nhất dù dữ liệu đã bị phân mảnh.
- Replication transparency: Dữ liệu có thể được nhân bản mà người dùng không biết.

1.2. Mô hình kiến trúc phân tán áp dụng

Đề tài áp dụng kiến trúc Client-Server phân tán với mô phỏng phân vùng logic. Các engines (Document, Graph, Hybrid) đại diện cho các node phân tán, trong khi METIS partitioning mô phỏng việc phân tán dữ liệu đồ thị trên nhiều node.

Section 2: Phân Tích Mô Hình Dữ Liệu Lai (Multi-Model Hybrid)

Hệ thống triển khai mô hình Multi-Model Database – một xu hướng hiện đại được ghi nhận bởi Angles & Gutierrez (2008) – kết hợp Document Model và Graph Model trong một hệ thống thống nhất.

Khía cạnh	Document Model	Graph Model	Hybrid (Đề tài)
Cấu trúc dữ liệu	JSON document	Nodes + Edges	JSON + NetworkX Graph
Truy vấn	BM25F full-text	BFS/DFS/Dijkstra	Combined scoring
Phân tán	Index sharding	METIS partitioning	Cả hai
Join mechanism	Không native	Không native	Join qua disease name
Điểm mạnh	Tìm kiếm ngữ nghĩa	Quan hệ phức tạp	Kết hợp cả hai

Điểm nối (join key) giữa hai mô hình là tên bệnh (disease name) – một khóa ngữ nghĩa xuất hiện trong cả trường initial_diagnosis của Document Store lẫn tên nút Disease trong Graph Store. Đây là thiết kế pragmatic phù hợp với quy mô prototype.

Section 3: Phân Tích Phân Vùng METIS Edge-Cut

3.1. So sánh Edge-Cut vs Vertex-Cut

Tiêu chí so sánh	Edge-Cut (Lựa chọn)	Vertex-Cut
Định nghĩa	Mỗi vertex thuộc đúng một partition	Mỗi edge thuộc đúng một partition
Communication cost	Chỉ trả phí cho cut edges	Phải replicate vertex metadata
Đồ thị phù hợp	Power-law thưa (sparse)	High-degree hub graph (dense)
Ứng dụng điển hình	Finite Element Methods, scientific graphs	Social networks, web graphs
Đồ thị y tế (200n, 800e)	Phù hợp – density thấp ~ 0.04	Không tối ưu
Edge-cut tỷ lệ mục tiêu	$ECR < 0.25$	Không áp dụng
Cài đặt METIS	Trực tiếp với gpmetis	Cần tiền xử lý

3.2. Chi tiết Recursive Bisection (3 bước)

- Bước 1 – Coarsening Phase:** Giảm kích thước đồ thị G xuống G' bằng Heavy Edge Matching (HEM). Với mỗi nút v chưa được ghép, tìm nút u liền kề có trọng số cạnh (v,u) lớn nhất để tạo supernode. Quá trình lặp cho đến khi $|G'| < \text{threshold}$ (thường 100-200 nút).
- Bước 2 – Initial Partitioning:** Phân chia đồ thị nhỏ G' thành k=4 phân vùng ban đầu bằng phương pháp Greedy Graph Growing hoặc Spectral Bisection. Ở bước này chất lượng phân vùng chưa tối ưu nhưng cân bằng tốt.
- Bước 3 – Uncoarsening & Refinement:** Mở rộng ngược từ G' về G, tại mỗi cấp độ áp dụng Kernighan-Lin (KL) refinement để di chuyển các nút biên giữa các phân vùng, tối thiểu hóa edge-cut trong khi giữ cân bằng tải.

3.3. Đo lường hiệu quả phân vùng

Chỉ số	Công thức	Giá trị mục tiêu	Ý nghĩa
Edge-Cut Ratio (ECR)	$ECR = E_{\text{cut}} / E_{\text{total}} $	< 0.25	Tỷ lệ cạnh bị cắt ngang partition
Balance Factor (*)	$\max(P_i) / \text{avg}(P_j)$	≤ 1.2	Độ cân bằng kích thước partition
Intra-Partition Density (*)	$ E_{\text{in}} / (V_{\text{in}} \times (V_{\text{in}} - 1) / 2)$	> 0.3	Mật độ cạnh trong partition
Communication Cost	$C = E_{\text{cut}} \times \text{latency}$	Tối thiểu hóa	Chi phí truy vấn cross-partition

(*) Các chỉ số này là mục tiêu thiết kế lý thuyết dựa trên tài liệu tham khảo Karypis & Kumar (1998). Trong bản prototype hiện tại, code tính ECR và Communication Cost, nhưng chưa implement tính toán trực tiếp Balance Factor và Intra-Partition Density.

Section 4: Xử Lý Truy Vấn Phân Tán

4.1. BM25F – Công thức và Justification

BM25F (Best Match 25 with Field-weighted scoring) là phát triển của Robertson & Zaragoza (2009) từ BM25 truyền thống. Lý do lựa chọn BM25F cho Document Engine:

- Hỗ trợ multi-field search trên 5 trường: chief_complaint, symptoms, medical_history, initial_diagnosis, full_text với trọng số bằng nhau (equal weighting), sử dụng MultifieldParser với OrGroup cho token matching.
- Saturation effect qua tham số k_1 (thường 1.2-2.0): ngăn TF rất cao dominate kết quả, quan trọng trong văn bản y tế.
- Length normalization qua b (thường 0.75): điều chỉnh cho sự khác biệt độ dài giữa chief_complaint ngắn và medical_history dài.

$$\text{score_BM25F}(q, d) = \sum_{t \in q} \text{IDF}(t) \times \left(\frac{\sum_f \text{tf}_f(t, d)}{\text{tf}_f(t, d) + k_1 \times (1 - b + b \times |d| / \text{avgdl})} \right)$$
 với $k_1=1.2$, $b=0.75$, $\text{boost}_f=1.0$ cho mọi trường.

4.2. BFS/DFS/Dijkstra – So sánh chi phí

Thuật toán	Độ phức tạp thời gian	Độ phức tạp không gian	Kết quả	Phù hợp khi
BFS	$O(V + E)$	$O(V)$	Đường đi ngắn nhất (theo hop)	Cần khoảng cách hop, đồ thị thưa
DFS	$O(V + E)$	$O(\text{depth})$	Duyệt sâu, không đảm bảo shortest	Khám phá toàn bộ subgraph
Dijkstra	$O((V+E) \log V)$	$O(V)$	Đường đi ngắn nhất có trọng số	Cạnh có trọng số (correlation weight)

Trong hệ thống, BFS được dùng mặc định để tính graph_importance vì đồ thị y tế có trọng số cạnh phản ánh strength of correlation, không phải distance. BFS đảm bảo hop count chính xác nhất.

Section 5: Phân Tích Chi Phí Ghép Nối

5.1. Mô hình chi phí 3 thành phần

Theo Özsu & Valduriez (2020), chi phí truy vấn phân tán gồm ba thành phần:

- I/O Cost: Chi phí truy xuất đĩa khi đọc documents từ Whoosh index và các node đồ thị từ phân vùng.
- CPU Cost: Chi phí tính toán gồm BM25F scoring, graph traversal, và join operations.

- **Communication Cost:** Chi phí giao tiếp mạng khi truy vấn cross-partition edges.

Mô hình chi phí cụ thể với số liệu prototype:

$$\begin{aligned}
 \text{IO_Cost} &= N_{\text{docs}} \times t_{\text{seek}} + \text{result_size} \times t_{\text{transfer}} \\
 &= 500 \times 0.1\text{ms} + k \times 0.01\text{ms} \quad (k = \text{result count}) \\
 \text{CPU_Cost} &= N_{\text{BM25F_ops}} \times t_{\text{cpu}} + N_{\text{graph_ops}} \times t_{\text{cpu}} \\
 &= (n_{\text{terms}} \times n_{\text{docs}}) \times 0.001\text{ms} + (V_{\text{visited}} + E_{\text{visited}}) \times 0.001\text{ms} \\
 \text{Comm_Cost} &= |E_{\text{cut}}| \times t_{\text{network}} \\
 &= \text{cut_edges} \times 1\text{ms} \quad (\text{local network assumption})
 \end{aligned}$$

5.2. So sánh 3 chiến lược Join

Chiến lược	IO Cost	CPU Cost	Comm Cost	Total (ước tính)	Khi nào dùng
Filter-After-Join	Cao (full scan)	Thấp	Thấp	$O(N \times G)$	N nhỏ, query mơ hồ
Index Nested-Loop	Trung bình	Trung bình	Trung bình	$O(k \times \log G)$	k vừa, G lớn
Hash Join	Cao (build phase)	Cao (hash build)	Thấp	$O(G + k)$	G và k đều lớn

5.3. Communication cost cho distributed query

Trong môi trường 4 phân vùng METIS, một truy vấn về "Cardiology" có thể cần truy cập nút trong các phân vùng khác nhau do cut edges. Chi phí này được tính:

$$\text{Comm_Cost}(\text{query}) = |\{\text{edges crossed by BFS traversal}\}| \times \text{network_latency}$$

Với $\text{ECR} < 0.25$, trung bình mỗi truy vấn BFS (depth=3) từ 1 Medical Field node sẽ crossing khoảng 15-30 edges, tương đương 15-30ms trên local network.

Section 6: Đánh Giá Theo Tiêu Chí Özsü & Valduriez (7 Tiêu Chí)

#	Tiêu chí	Mô tả tiêu chí	Triển khai trong đề tài	Đánh giá
1	Distribution Transparency	Che giấu sự phân tán khỏi người dùng	API /api/query ẩn hoàn toàn partition logic	Đạt
2	Replication Transparency	Che giấu nhân bản dữ liệu	Không nhân bản trong prototype; fallback METIS đảm bảo availability	Một phần
3	Fragmentation Transparency	Che giấu sự phân mảnh dữ liệu	METIS partitions hoàn toàn trong suốt với người dùng	Đạt
4	Design Autonomy	Mỗi site có schema độc lập	Document Engine và Graph Engine có schema độc lập, join qua semantic key	Đạt

5	Query Optimization	Tối ưu hóa truy vấn phân tán	3 chiến lược join với phân tích chi phí; chọn Hash Join khi $k > 50$	Đạt
6	Transaction Management	ACID trong môi trường phân tán	Read-only queries; không cần distributed transaction trong scope đề tài	N/A
7	Performance & Scalability	Hiệu năng khi scale dữ liệu	Whoosh index $O(\log N)$; METIS partition giảm graph traversal cost	Đạt

Section 7: Hạn Chế và Hướng Phát Triển

7.1. Hạn chế hiện tại

- Dữ liệu tổng hợp: 500 bệnh nhân là dataset nhỏ; BM25F precision/recall chưa được validate trên dữ liệu thực tế.
- Phân tán logic: METIS partitioning được mô phỏng trên một máy; không có thực sự network overhead.
- Không có transaction management: Hệ thống chỉ hỗ trợ read-only queries; không có distributed ACID.
- METIS dependency: Nếu METIS binary không khả dụng, fallback hash partitioning có chất lượng thấp hơn đáng kể.
- Join key fragility: Join qua disease name string dễ bị lỗi nếu chuẩn hóa tên bệnh không nhất quán.

7.2. Hướng phát triển

- Triển khai thực tế trên nhiều node: Sử dụng Apache Kafka hoặc gRPC để tạo distributed engine thực sự.
- Tích hợp LLM embedding: Thay BM25F bằng vector similarity search (FAISS) kết hợp semantic embedding để tăng recall.
- Graph Neural Network: Sử dụng GNN để tính graph_importance thay vì chỉ dựa trên hop count.
- Temporal analysis: Thêm chiều thời gian vào đồ thị bệnh để phân tích xu hướng dịch bệnh.
- Federated Learning: Mỗi bệnh viện giữ dữ liệu riêng, chỉ chia sẻ model weights – đảm bảo quyền riêng tư y tế.

Tài Liệu Tham Khảo

- [1] M. T. Özsu and P. Valduriez, Principles of Distributed Database Systems, 4th ed. Springer, 2020. ISBN 978-3-030-26253-2.
- [2] G. Karypis and V. Kumar, "A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs," SIAM Journal on Scientific Computing, vol. 20, no. 1, pp. 359–392, 1998.
- [3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.
- [4] R. Angles and C. Gutierrez, "Survey of Graph Database Models," ACM Computing Surveys, vol. 40, no. 1, Article 1, 2008.

- [5] C. J. Date, "Twelve Rules for a Distributed Data Base," Computerworld, vol. 21, no. 23, pp. 75–82, 1987.
- [6] F. Petroni et al., "Language Models as Knowledge Bases?" EMNLP 2019, pp. 2463–2473.
- [7] Whoosh Library Documentation, <https://whoosh.readthedocs.io/en/latest/>
- [8] NetworkX Documentation, <https://networkx.org/documentation/stable/>

Trần Hữu Trung – N23DCCN200